Datenbanken

Relationale Algebra

Relationales Modell

Domäne	Endliche Menge von Werten desselben Typs ($\approx$ Datentyp)
Attribut	Eigenschaft mit eindeutigem Namen, genau eine Domäne $\text{dom}(A)$
n-Tupel	Menge von n zusammengehörigen Attributwerten
Relationsschema	(Format, Heading, Relationsvariable) Menge von Attributen
Relation	Schema + Menge von Tupeln dazu (Ausprägung $\text{ext}(R)$)

- Relation = endliche **Menge**: keine Duplikate, keine Tupel-Reihenfolge; Attributreihenfolge egal
- **Notation:** Schema $R(A_1, A_2, ..., A_n)$; Tupelmenge $\{\langle a, b, c\rangle, ...\}$ (Format immer angeben)
- **Äquivalenz** $R_1 \sim R_2$: durch Umordnen der Attribute ineinander überführbar (gleiche Domänen/Bedeutung)

Schlüssel

- **Schlüssel:** nicht-leere Attributmenge, identifiziert Tupel eindeutig
- **Schlüsselkandidat:** minimale Schlüsselmenge; jede Relation hat mind. einen (Format selbst)
- **Surrogatschlüssel:** künstlicher Schlüssel, wenn nichts passt
- **Primärschlüssel (PK):** gewählter Kandidat, auf den ein Fremdschlüssel zeigt
- **Fremdschlüssel (FK):** Attributkombination, die auf PK einer anderen Relation zeigt; PK-FK explizit definieren (gleicher Name genügt nicht)

Grundbegriffe

- **RA:** Operatoren + Regeln zum Rechnen mit Relationen; Relationen = Operanden; Ausdrücke = Anfragen (queries)
- **Prädikat:** berechenbarer Ausdruck $\rightarrow$ wahr/falsch
- **Kriterien Abfragesprache:** Ad-hoc-Formulierung, deklarativ („was“, nicht „wo“), optimierbar (wenige Operatoren, Optimierungsregeln), **abgeschlossen** (Ergebnis wieder Relation $\rightarrow$ schachtelbar), mengenorientiert (nicht tuple-at-a-time), sicher (keine Endlosschleife/unendliches Ergebnis)

Selektion σ (entfernt Tupel)

$R' = \sigma_P(R)$, unär

- Prädikat P: logischer Ausdruck aus Attributen/Konstanten ($\wedge, \vee, \neg, =, <, \geq, ...$), für jedes Tupel geprüft
- Ergebnis: **gleiches Schema**, Tupel mit P wahr; kann leer sein (z. B. $\sigma_{1=0}(r) \rightarrow$ nur Schema, keine Tupel)
- Z. B. $\sigma_{\text{City = 'Zürich'} \wedge \text{ Discount } \geq 0.15}(\text{Customers})$
- Kommutativ: $\sigma_{\Phi_1}\big(\sigma_{\Phi_2}(r)\big) = \sigma_{\Phi_2}\big(\sigma_{\Phi_1}(r)\big) = \sigma_{\Phi_1 \wedge \Phi_2}(r)$

Projektion π (entfernt Attribute)

$R' = \pi_{A_1,...,A_k}(R)$, unär

- Schema = Attributliste; Projektion auf nicht vorhandene Attribute $\rightarrow$ **Fehler**
- **Erweitert:** $\pi_{E \rightarrow x,...}(R)$, Ausdruck E aus Attributen, Konstanten, arithm./String-Operatoren, Funktionen; Ergebnis heisst x. Z. B. $\pi_{3 \cdot A \rightarrow X, B+C \rightarrow D, C}(R)$, $\pi_{\text{SQR}(A) \rightarrow X}(R)$
- Duplikate werden entfernt
  RDBMS/SQL entfernen Duplikate nicht automatisch
- **Nicht** vertauschbar: $\pi_A(\pi_B(r)) \neq \pi_B(\pi_A(r))$ i. A.

Umbenennung ρ

- Gleiche Attributnamen in versch. Relationen qualifizieren: $R.A$, $S.A$
- $\rho_{\text{neu}}(R)$ (auch `R AS neu`): Relation umbenennen, z. B. für Auto-Join
- $\rho_{S(D,E)}(R(A, B))$: $R.A \rightarrow S.D$, $R.B \rightarrow S.E$

Kartesisches Produkt $\times$

$R' = R \times S$, binär

- Jedes Tupel aus R mit jedem aus S kombiniert $\rightarrow$ $|R \times S| = |R| \cdot |S|$
- Schema: R konkateniert mit S; gleichnamige Attribute bleiben beide $\rightarrow$ qualifizieren ($R.B$, $S.B$) oder umbenennen
- In SQL nicht kommutativ: $R \times S \neq S \times R$ (Schema-Reihenfolge; per π überführbar)

Theta-Join $\bowtie_P$ (allg. Verbund)

$R \bowtie_P S = \sigma_P(R \times S)$

- Join-Prädikat P frei: beliebiger log. Ausdruck mit θ-Vergleichen, $\theta \in \{=, <, >, \leq, \geq, \neq\}$
- Schema wie Kreuzprodukt (alle Attribute, ggf. qualifizieren $R.B$, $S.B$)
- Verallgemeinerung des Natural Join, in Praxis Normalfall

Verbundvarianten (-Joins)

Natural	$R \bowtie S$: Gleichheit in allen gemeinsamen Attributen (Name, Domäne, Bedeutung), gemeinsame nur einmal im Schema. Keine gemeinsamen Attribute $\rightarrow$ Kreuzprodukt.
Equi	P nur Gleichheiten, mit $\wedge$ verknüpft. Natural Join $\subset$ Equi-Join (Equi-Join: Attribute dürfen versch. heissen, beide bleiben im Schema)
Auto	$R_1 \bowtie R_1$, braucht Umbenennung ρ
Cross	$R_1 \times R_2$ (kartesisches Produkt)
Semi	$\pi_{r_1..r_n}(R_1 \bowtie R_2)$: Join, projiziert auf Attribute von R_1 (oder R_2)

Mengenoperatoren $\cup$, $\cap$, $\setminus$

Binär; R, S müssen **gleiches Schema** haben. Ergebnis: gleiches Schema, ohne Duplikate.

Op.	Tupel in	kommutativ
$R \cup S$	R ODER S	ja
$R \cap S$	R UND S	ja
$R \setminus S$ $(R - S)$	R ABER NICHT S	nein

Bag-Algebra (Multimengen)

SQL arbeitet mit **Bags**: identische Tupel nicht unterscheidbar, aber Anzahl Vorkommen = **Multiplizität** m.

σ, $\times$, $\bowtie$	Wie RA, Duplikate wie normale Tupel
π	Wie RA, Duplikate **nicht** entfernen
$\cup$ bag union	$\max(m_R, m_S)$
$\sqcup$ bag concatenation	$m_R + m_S$
$\cap$	$\min(m_R, m_S)$
$\setminus$	$\max(m_R - m_S, 0)$
δ Duplikatelimination	Entfernt Duplikate: Bag $\rightarrow$ Relation

Outer Joins $\bowtie$, $\bowtie$, $\bowtie$

Alle Tupel des linken ($\bowtie$ left), rechten ($\bowtie$ right) bzw. beider ($\bowtie$ full) Operanden ins Resultat; ohne Join-Partner $\rightarrow$ fehlende Werte mit **NULL** auffüllen.

- NULL ist **kein Wert**, sondern Indikator für fehlenden Wert; verursacht Probleme $\rightarrow$ möglichst vermeiden

$L(A, B, C)$	$R(C, D, E)$	$L \bowtie R$
a_1, b_1, c_1 a_2, b_2, c_2	c_1, d_1, e_1 c_3, d_2, e_2	a_1, b_1, c_1, d_1, e_1 a_2, b_2, c_2, NULL, NULL NULL, NULL, c_3, d_2, e_2

$\bowtie$: nur Zeilen 1–2; $\bowtie$: Zeilen 1 und 3; $\bowtie$: nur Zeile 1.

Aggregation & Gruppierung

- **Aggregatfunktion:** fasst Attributwerte mehrerer Tupel zusammen: Summe, Durchschnitt, Min, Max, Anzahl
- Über alle Tupel oder pro **Gruppe**: Multimenge von Tupeln mit gleichen Werten im Gruppierungskriterium (Attributkombination), `GROUP BY a` $\rightarrow$ ein Tupel pro Wert von a mit $f(a)$
- Aggregation, Gruppierung, Sortierung: nicht algebraisch

Äquivalenzen

Kommutativität	$\sigma_\Phi(\sigma_\psi(r)) = \sigma_\psi(\sigma_\Phi(r))$ $\pi_A(\sigma_\Phi(r)) = \sigma_\Phi(\pi_A(r))$, falls Φ nur Attr. aus A $r \bowtie s = s \bowtie r$ (Format verschieden)
Assoziativität	$r \bowtie (s \bowtie t) = (r \bowtie s) \bowtie t$
Idempotenz	$\pi_A(\pi_C(r)) = \pi_A(r)$, falls $A \subseteq C$ $\sigma_\Phi(\sigma_\psi(r)) = \sigma_{\Phi \wedge \psi}(r)$
Distributivität	$\pi_A(r \cup s) = \pi_A(r) \cup \pi_A(s)$ $\sigma_\Phi(r \cup s) = \sigma_\Phi(r) \cup \sigma_\Phi(s)$ $\sigma_\Phi(r \bowtie s) = \sigma_\Phi(r) \bowtie s$, falls Φ nur Attr. von r $\pi_{A,B}(r \bowtie s) = \pi_A(r) \bowtie \pi_B(s)$, falls Joinattr. $J \subseteq A \cap B$ $r \bowtie (s \cup t) = (r \bowtie s) \cup (r \bowtie t)$

By Justin Iven Müller (2026)
github.com/JustinIven/zhaw-cheatsheets